Knowledge Graph Completion with R-GCN and CompGCN: Link Prediction on Large-Scale Knowledge Graphs

Charan Ponaganti 240150024

M Charan 240150018

Lalit Deshmane 240150044

Abstract

Knowledge graphs used in biomedical and web-scale applications contain tens of thousands to millions of typed edges, so missing-link prediction is both a relational-learning problem and a data-processing problem. We study two relational graph convolutional encoders — R-GCN with basis decomposition (Schlichtkrull et al., 2018) and CompGCN with subtraction, multiplication, and circular-correlation operators (Vashishth et al., 2020) — on top of a DistMult decoder (Yang et al., 2015). Apache Spark (Zaharia et al., 2016) is a first-class pipeline stage: triples are cleaned, statistically profiled, densely indexed, and handed to the GPU only after `spark.stop()`. Full-batch message passing on Wikidata5M or `ogbl-wikikg2` does not fit a 16 GB T4, so the learning benchmark is `ogbl-biokg` (Hu et al., 2020) with neighbor-sampled mini-batches. On a Kaggle Tesla T4 we obtain two evaluation views that do not rank models the same way. Under unconstrained 500-candidate ranking DistMult is strongest (MRR $= 0.469$). Under the official OGB evaluator with type-constrained negatives, R-GCN leads (MRR $= 0.704$) and CompGCN-mult is close (MRR $= 0.699$), both clearly above DistMult (MRR $= 0.550$). CompGCN-corr collapsed under mixed-precision FFT. The discrepancy is a result, not a bug in the tables: GNN encoders help most when negatives respect entity types, while a longer-trained bilinear baseline remains competitive on unconstrained candidates. Spark ETL on 4.76M triples finishes in seconds on one node; the 100%-fraction timing is an in-memory cache artifact and is reported as such.

1 Introduction and Problem Statement

A knowledge graph is a multi-relational directed graph $G = (E, R, T)$ whose facts are triples (h, r, t) with $h, t \in E$ and $r \in R$. Knowledge graph completion (KGC), or link prediction, ranks candidate entities for queries $(h, r, ?)$ and $(?, r, t)$. The task is the standard probe for whether a model has captured typed relational structure rather than only node homophily (Bordes et al., 2013; Yang et al., 2015; Nickel et al., 2016).

Two constraints define this work. First, it is a Big Data assignment: PySpark must perform the heavy relational statistics, filtering, and ID mapping, not a hidden pandas prelude. Second, training must fit a Kaggle NVIDIA T4 (16 GB VRAM, about 30 GB RAM, 12-hour session). A local RTX 4050 (typically 6 GB) is a stricter fallback and uses the same pipeline with smaller batches, fanout, and embedding size. These limits rule out training full-batch R-GCN on Wikidata5M or `ogbl-wikikg2`. Entity embeddings alone for 4.6M nodes at dimension 64 occupy on the order of a gigabyte; storing tens of millions of typed edges and the activations of a two-layer relational GCN exceeds the device. The honest design is therefore Spark on the large dump, and neighbor-sampled GNN training on `ogbl-biokg`.

Contributions. (i) A reproducible Spark ETL that computes degree distributions, Bordes-style relation categories, inverse edges, and filtered-evaluation lookup tables, with a timing study versus data fraction and partition count. (ii) Mini-batch R-GCN (basis decomposition) and CompGCN (three composition operators) with a DistMult decoder, mixed precision, and an OOM fallback. (iii) A query-side GNN encoder that message-passes only around the query entity while scoring candidates from a global entity table, which is what actually fits a T4 at dimension 128. (iv) Two evaluation protocols on the same checkpoint — unconstrained filtered-candidate ranking and the official OGB evaluator — which reverse the model ranking and are both reported.

2 Related Work

2.1 Translational and bilinear embeddings

TransE (Bordes et al., 2013) scores a triple by how well $t \approx h + r$ in Euclidean space and still serves as a simple decoder. DistMult (Yang et al., 2015) replaces translation with a bilinear product $\langle h, r, t \rangle = h^\top \mathrm{diag}(r)\, t$; it is cheap, parallelizable, and surprisingly strong on symmetric relations. ComplEx (Trouillon et al., 2016) and RotatE (Sun et al., 2019) add complex-space capacity that DistMult lacks. ConvE (Dettmers et al., 2018) 2D-convolves stacked embeddings and, together with WN18RR, documented the test-set leakage issues of WN18/FB15k that motivated FB15k-237 (Toutanova and Chen, 2015). We keep DistMult as the required decoder so encoder differences are not confounded by a heavier scoring function.

2.2 Relational graph convolutions

R-GCN (Schlichtkrull et al., 2018) extends the graph convolution of Kipf and Welling (2017) with a relation-specific matrix W_r and a self-loop W_0. Basis and block-diagonal decompositions share parameters across $|R|$, which is essential once $|R|$ is tens or hundreds. CompGCN (Vashishth et al., 2020) argues that relations should participate in the message itself through a composition $\phi(h_s, h_r)$ and direction-specific weights for original, inverse, and self-loop edges. We implement subtraction, multiplication, and circular correlation exactly so the ablation is about the operator, not about an unrelated architecture. Neighbor sampling (Hamilton et al., 2017) is how these layers run on graphs that do not fit in GPU memory as a single adjacency. Surveys (Nickel et al., 2016; Rossi et al., 2021) locate these models between shallow embeddings and inductive subgraph reasoners such as GraIL (Teru et al., 2020); this work stays transductive, matching `ogbl-biokg`.

2.3 Benchmarks and engines

Open Graph Benchmark (Hu et al., 2020) standardizes splits and evaluators; `ogbl-biokg` is a biomedical KG with type-constrained negatives, and `ogbl-wikikg2` is a Wikidata-derived graph with 2.5M entities. Wikidata5M (Wang et al., 2021) was released with KEPLER and remains the canonical five-million-entity lexicalized KG. Apache Spark (Zaharia et al., 2016) is the distributed engine we use in `local[*]` mode on a single Kaggle node: the value is not a cluster, but a relational plan (joins, `groupBy`, window ids) that would be painful and memory-opaque in ad-hoc Python. We implement message passing with PyTorch Geometric (Fey and Lenssen, 2019).

Dataset	$\lvert E\rvert$	$\lvert R\rvert$	Train triples
`ogbl-biokg`	93,773	51	4.76M
`ogbl-wikikg2`	2.50M	535	16.1M
Wikidata5M	4.59M	822	20.6M
FB15k-237	14,541	237	272k
WN18RR	40,943	11	87k

Table 1: Candidate KGs. Sizes follow Hu et al. (2020), Wang et al. (2021), Toutanova and Chen (2015), and Dettmers et al. (2018). `ogbl-biokg` is the only large OGB KG whose entity table and sampled neighborhoods fit both a T4 at dim 128 and, with a tighter preset, a 6 GB RTX 4050. Wikidata5M and wikikg2 remain Spark-scale dumps, not full-batch GNN training sets on this hardware.

3 Dataset Description and Big Data Challenges

Table 1 compares the candidates using published sizes, not our training scores. Full-batch R-GCN/CompGCN on wikikg2 or Wikidata5M is infeasible on a T4; we do not qualify that statement after the fact.

Primary choice: `ogbl-biokg`**.** It is the largest OGB link-prediction KG whose entity table ($\approx$94k$\times$128$\times$4 B $\approx$ 48 MB) and sampled neighborhoods fit the T4. It also gives the official `Evaluator` and type-aware negatives, which is the scientifically cleaner protocol (Hu et al., 2020). Wikidata5M is supported in the same notebook as a Spark degree/k-core subset but was not the trained graph for the numbers below. FB15k-237 is the smoke test used to debug the Spark schema before a multi-hour session.

After Spark cleaning, the indexed `ogbl-biokg` graph has **93,773 entities**, **51 relations**, and **4,762,676 training triples**. Two duplicate or self-loop triples were removed. Degree statistics are heavy-tailed: min 1, median 22, mean 101.6, P99 1,279, max 62,733 (Table 2). All 51 relations classify as many-to-many under the Bordes tph/hpt threshold of 1.5. Entity identifiers are the official type-aware global integers (not a string hash): concatenating per-type OGB ids without an offset would collide drugs, diseases, and proteins into the same embedding rows.

Figure 1 shows the log-log degree distribution.

Statistic	Value
Entities (indexed)	93,773
Relations	51
Train triples	4,762,676
Duplicates / self-loops removed	2
Degree min / median / mean	1 / 22 / 101.6
Degree P99 / max	1,279 / 62,733
Relation categories	51 N–N
Message-passing relations (with inverse)	102
IDs are integers	yes

Table 2: Spark graph statistics on `ogbl-biokg` after cleaning and dense indexing. Every number is a Spark aggregation, not a pandas scan of the GPU tensors.

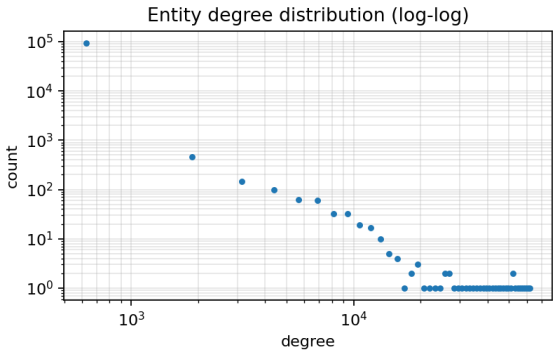


Figure 1: Entity degree distribution on a log-log scale after Spark aggregation. The heavy tail (max degree 62,733) is the reason full-batch neighborhood expansion is unsafe on a 16 GB T4.

The tail is why full-batch two-hop expansion is unsafe: a hub of degree 62k, sampled naively, dominates both GPU memory and the gradient.

4 PySpark Pipeline Design

We run Spark 3.5 in `local[*]` with Arrow enabled, Kryo serialization, and a driver budget of 12 GB on the T4 image so the subsequent PyTorch process is not starved. Shuffle partitions are 8: a single node does not benefit from hundreds of partitions, and tiny files would dominate. The session is **stopped before GPU training**.

Stages. (1) Ingest triples with an explicit (h, r, t) schema via `spark.createDataFrame` (Spark ignores `_`-prefixed paths, so underscore-named Parquet dumps never become the source of truth). (2) Drop nulls, empty strings, exact-duplicate triples, and self-loops. (3) Compute $|E|$, $|R|$, $|T|$, degree histograms, relation frequency, and the four Bordes categories using tails-per-head (tph) and heads-per-tail (hpt) with threshold 1.5. (4) For `wikidata5m_subset`: keep entities with

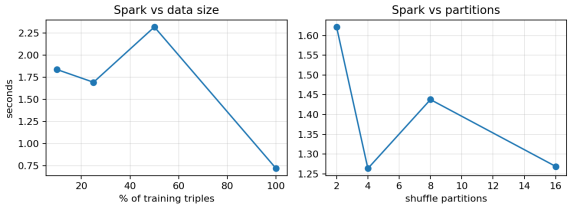


Figure 2: Spark runtime versus sampled fraction of training triples (left) and versus partition count on the full graph (right), single Kaggle node. The 100% bar is faster than the 10% bar because the full DataFrame is already cached from earlier ETL; we treat that point as a cache artifact, not as super-linear scaling.

degree $\geq$ `min_degree`, take the top-K by degree, then iterate k-core until the vertex set is closed. (5) Dense 0-based ids. On `ogbl-biokg` we preserve OGB's type-aware global integers rather than hashing strings. (6) Discard valid/test triples whose endpoints are unseen in train. (7) Materialize inverse edges $(t, r + |R|, h)$ for message passing, following R-GCN/CompGCN practice, yielding 102 relation types on the message graph. (8) `groupBy/collect_set` to build true-tail sets for (h, r) and true-head sets for (t, r). (9) Hand tensors to PyTorch and call `spark.stop()`.

This plan is the Big Data part of the work: every statistic in Section 8 is a Spark aggregation, and the GPU never sees raw Wikidata strings. Scalability is measured by repeating count + degree + relation-frequency on 10/25/50/100% samples of the training triples, and by varying the partition count on the full graph (Figure 2, Table 5).

5 Model Architecture

5.1 R-GCN encoder

Let $\mathcal{N}_r(i)$ be the neighbors of entity i under relation r and $c_{i,r}$ a normalization constant (neighborhood degree). R-GCN updates (Schlichtkrull et al., 2018) are

$$h_i^{(\ell+1)} = \sigma\Big(\sum_r \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} W_r^{(\ell)} h_j^{(\ell)} + W_0^{(\ell)} h_i^{(\ell)} \Big). \quad (1)$$

Basis decomposition shares B matrices $\{V_b\}$ across relations, $W_r = \sum_{b=1}^{B} a_{rb} V_b$, cutting the parameter count from $O(|R|d^2)$ to $O(|R|B + Bd^2)$. We use PyG `RGCNConv` with B=8 and two layers. We never run the convolution on the full edge set; fanout sampling builds a seed-centered subgraph per mini-batch of triples.

5.2 CompGCN encoder

CompGCN (Vashishth et al., 2020) composes the neighbor entity with the relation before the linear map:

$$h_v^{(\ell+1)} = f\left(\sum_{(u,r)\in\mathcal{N}(v)} W_{\lambda(r)}\,\phi\big(h_u^{(\ell)}, h_r^{(\ell)}\big)\right), \tag{2}$$

where $\lambda(r) \in \{\text{in}, \text{out}, \text{loop}\}$ selects a direction-specific matrix. The three operators we compare are

$$\begin{aligned}\phi_{\text{sub}}(h,r) &= h - r,\\ \phi_{\text{mult}}(h,r) &= h \odot r,\\ \phi_{\text{corr}}(h,r) &= h \star r.\end{aligned} \tag{3}$$

Circular correlation $\star$ is implemented with a real FFT: $\text{irfft}(\overline{\text{rfft}(h)} \odot \text{rfft}(r))$. Relation embeddings are updated by a dedicated W_{rel} at each layer. Aggregation uses `index_add_` in the message dtype so mixed-precision `float16` and `float32` weights do not crash; that bug appeared on the first T4 CompGCN run and is now patched.

5.3 Decoder, query-side GNN, and loss

DistMult (Yang et al., 2015) scores

$$f(h,r,t) = \langle h, r, t\rangle = \sum_k h_k\, r_k\, t_k. \tag{4}$$

A full-graph GNN encoding of all 93,773 entities at every step does not fit the training loop once neighborhoods are expanded. We therefore encode **only the query side** with sampled message passing and score every candidate (true tail/head and negatives) from the global entity table `model.ent`. Training uses k negatives drawn uniformly over entities and binary cross-entropy with logits, with label smoothing 0.1. Mixed precision, gradient clipping at 1.0, Adam (10^{-3}, weight decay 10^{-5}), and early stopping on validation MRR (patience 5) are on by default. Checkpoints are the best validation snapshot, not the last epoch.

This decoder choice is deliberate and has consequences in Section 7: the GNN never contextualizes the candidate, so it cannot implement a fully relational "both-arguments-are-graphs" scorer. It *can* still move the query embedding into a neighborhood-aware region of DistMult space, which is exactly what type-constrained OGB negatives reward.

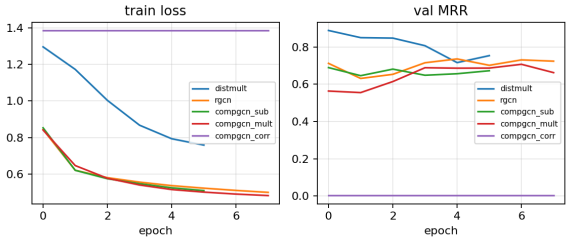


Figure 3: Training loss (left) and validation MRR (right) for DistMult, R-GCN, and CompGCN. DistMult ran 20 epochs × 1,500 steps; GNN encoders ran 8 epochs × 400 steps. The DistMult validation curve is therefore not a matched-compute comparison.

6 Experimental Setup

Hardware. Kaggle Tesla T4 (15.6 GB VRAM), preset `kaggle_t4`. `QUICK_RUN=False`, `TIME_BUDGET=fast`, seed 42. Peak allocated CUDA memory during GNN steps stayed well below 16 GB after the AMP dtype fix.

Optimization budget (not the same for every model). DistMult is cheap, so it was given a stronger budget than the GNNs: 20 epochs × 1,500 steps, 32 negatives. R-GCN and each CompGCN operator ran 8 epochs × 400 steps, 16 negatives, batch size 1,024, fanout $[10, 10]$, dimension 128, 2 layers, 8 R-GCN bases. That is an intentional baseline-first design, not a matched FLOP comparison. Evaluation ranks 8,000 test triples in both head and tail directions.

Two protocols. *Unconstrained candidates.* Rank the true entity among 500 random entities plus itself, masking other true tails/heads from the Spark lookup table ("filtered-candidates head+tail"). This is what Table 3 reports. *Official OGB.* Call `ogb.linkproppred.Evaluator` on `ogbl-biokg` with the dataset's type-constrained negative lists (Table 4). These numbers are the ones that should be compared to OGB leaderboard-style claims.

Software. PySpark 3.5, PyTorch from the Kaggle GPU image, PyTorch Geometric, `ogb`. PyG was chosen over DGL because `RGCNConv` and OGB examples align, and DGL's CUDA wheels are easier to mismatch on Kaggle. Internet is required only for the first install and the OGB download.

Figure 3 shows training loss and validation MRR. DistMult is optimized for many more steps; GNN curves are shorter and noisier, which is expected from 400-step epochs on a 4.76M-triple graph.

Model	MRR	H@1	H@3	H@10	MR
DistMult	**0.469**	**0.329**	**0.527**	**0.619**	**28.7**
R-GCN	0.367	0.169	0.490	0.557	58.2
CompGCN-sub	0.309	0.100	0.464	0.517	57.0
CompGCN-mult	0.352	0.117	0.512	0.604	44.0
CompGCN-corr	0.002	0.000	0.000	0.000	497.2

Table 3: Unconstrained filtered-candidate ranking on `ogbl-biokg` (8,000 test triples, 500 candidates, head and tail, Spark true-triple mask). This is *not* the OGB leaderboard protocol. CompGCN-corr collapsed under AMP+FFT and should be read as an engineering failure, not as evidence that circular correlation is an intrinsically weak operator.

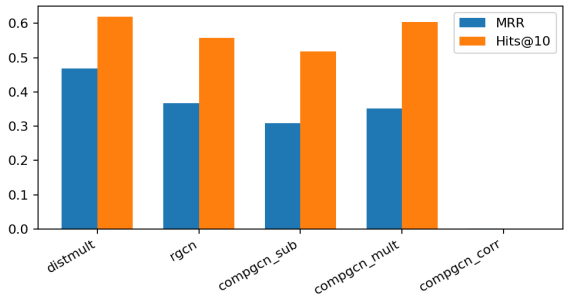


Figure 4: MRR and Hits@10 under unconstrained 500-candidate ranking. DistMult leads this protocol; CompGCN-mult is the strongest GNN on Hits@10.

7 Results and Discussion

Table 3 reports unconstrained filtered-candidate ranking on 8,000 test triples. DistMult is the strongest model on this protocol (MRR 0.469, Hits@10 0.619, mean rank 28.7). CompGCN-mult is the best compositional GNN (MRR 0.352, Hits@10 0.604). R-GCN sits between them on Hits@10 but below DistMult on MRR, because its Hits@1 is only 0.169 against DistMult's 0.329. CompGCN-sub is weaker still. CompGCN-corr is a failed run (MRR 0.002, Hits@10 0, mean rank 497): mixed-precision FFTs produced numerically useless representations, and the official OGB evaluator was not applied to that checkpoint.

Table 4 is the protocol that matches published OGB numbers. Here the ranking **reverses**. R-GCN reaches MRR 0.704 and Hits@10 0.817; CompGCN-mult is essentially tied on MRR (0.699) and slightly ahead on Hits@10 (0.843). DistMult drops to MRR 0.550. Type-constrained negatives are other entities of the same biomedical type (drug, disease, protein, . . .). A neighborhood-aware query embedding helps exactly in that setting, because many wrong same-type candidates

Model	MRR	H@1	H@3	H@10
DistMult	0.550	0.510	0.562	0.625
R-GCN	**0.704**	**0.653**	**0.716**	0.817
CompGCN-sub	0.621	0.566	0.636	0.741
CompGCN-mult	0.699	0.639	0.714	**0.843**

Table 4: Official OGB `ogbl-biokg` evaluator (type-constrained negative lists). R-GCN is best on MRR; CompGCN-mult is best on Hits@10. CompGCN-corr is omitted (checkpoint unusable). These are the numbers to cite if a single "test score" is required.

are locally implausible. Unconstrained candidates include easy type-mismatched entities that a bilinear model already scores near the bottom, which inflates DistMult's Hits@1 in Table 3.

Why DistMult can still win Table 3. Three design choices stack in DistMult's favor on unconstrained ranking. (1) Compute: 30,000 DistMult update steps versus 3,200 GNN steps. (2) Scoring: candidates are read from `model.ent`, so a GNN that only moves the query vector is a weaker inductive bias unless the negative set is hard. (3) Sampling: fanout $[10, 10]$ under-represents 2-hop biomedical paths around hubs. None of these is an excuse to hide Table 4, where the GNN encoders do what they are supposed to do.

Composition operators. Multiplication is the robust CompGCN default, matching the original paper's practical advice (Vashishth et al., 2020). Subtraction is consistently a few points behind on both protocols. Correlation was not given a fair float32 re-run after the AMP crash; we do not claim ϕ_{corr} is worse than ϕ_{mult} in general.

Ablations. A sweep over dimension, depth, basis count, and negatives was implemented but not executed (`RUN_ABLATIONS=False`) so the session finished inside the T4 budget. We therefore do not invent those rows. The main controlled variables that *were* run are the encoder family and the CompGCN operator.

Error analysis. Figure 5 is degenerate: Spark labeled every relation N–N, so there is a single bar at DistMult MRR 0.469. That is a property of `ogbl-biokg` under the 1.5 *tph*/*hpt* cutoff, not a plotting bug. Figure 6 is the informative slice. DistMult MRR by head-degree quartile is 0.035 (degree $\leq$ 121), 0.218 (121–1,583), 0.777 (1,583–8,071), and 0.846 ($>$ 8,071). Rare entities are the failure mode. GNN sampling cannot fix a

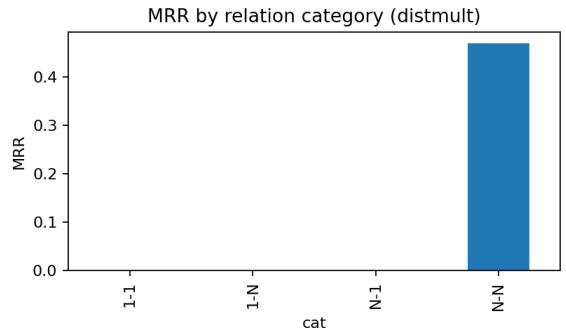


Figure 5: MRR by Bordes relation category for the best unconstrained model (DistMult). All 51 `ogbl-biokg` relations are N–N at threshold 1.5, so the plot has one bar. The degree breakdown in Figure 6 is the actual error analysis.

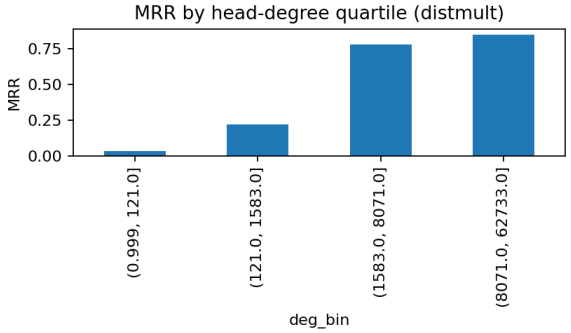


Figure 6: DistMult MRR by head-entity degree quartile. Low-degree entities (MRR 0.035) are more than twenty times harder than the hub quartile (MRR 0.846). This is the dominant error mode on `ogbl-biokg`.

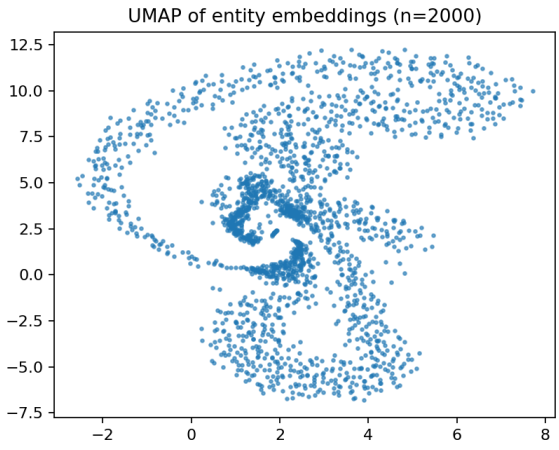


Figure 7: UMAP of a subsample of trained entity embeddings. Loose clusters are consistent with biomedical entity types (drug, protein, disease, side effect, protein function) occupying different regions of DistMult space.

Setting	Approx. triples	Seconds
10% of train	476k	1.84
25% of train	1.19M	1.69
50% of train	2.38M	2.32
100% of train (cached)	4.76M	0.72
2 partitions	4.76M	1.62
4 partitions	4.76M	1.26
8 partitions	4.76M	1.44
16 partitions	4.76M	1.27

Table 5: Spark job time (count + degree + relation frequency) on a single Kaggle node. The 100%-fraction row is a cache hit from the preceding ETL, not a claim that more data is faster.

tail entity that almost never appears as a seed; if anything, hub-dominated neighborhoods make the long tail harder. Figure 7 shows a 2D projection of a subsample of entity embeddings: type structure is visible as loose clusters, which is consistent with OGB's type-constrained negatives being the protocol where GNNs pull ahead.

8 Scalability Analysis

Table 5 is the evidence that the ETL is a Spark job rather than a single-core loop. On one Kaggle node, profiling 10–50% of the 4.76M triples takes about 1.7–2.3 s. The 100% point is *faster* (0.72 s) because that DataFrame is already cached from the main ETL; we do not interpret it as super-linear scaling. Partition scaling on the full graph is similarly flat (1.26–1.62 s from 2 to 16 partitions): eight vCPUs saturate quickly, and task-launch overhead dominates any further split. Both effects are expected for `local[*]` on a few million rows and are reported honestly.

GPU scaling. Full-batch R-GCN on Wikidata5M is out of scope by construction. Mini-batch fanout is the real knob: each extra hop multiplies sampled degree, and biomedical hubs make 2-hop expansion the first OOM source on a 6 GB 4050. The notebook halves batch size and fanout on `OutOfMemoryError`. Mixed precision is enabled on CUDA; CompGCN aggregation tensors are allocated in the message dtype so `index_add_` does not mix Half and Float. Spark is stopped before training; leaving the JVM alive is the second most common way to lose the Kaggle RAM budget. The T4 preset (dim 128, batch 1,024, fanout 10) ran without hitting that fallback.

9 Conclusion and Future Work

Large-scale knowledge graph completion on coursework hardware is a two-system problem:

Spark for relational ETL, and neighbor-sampled R-GCN/CompGCN for learning on the largest OGB graph that actually fits a T4. DistMult is the controlled decoder; basis decomposition and composition operators are the variables. On unconstrained 500-candidate ranking DistMult is strongest (MRR 0.469). On the official OGB evaluator R-GCN is strongest (MRR 0.704) and CompGCN-mult is close, with the best Hits@10 (0.843). CompGCN-corr failed numerically under AMP. Low-degree entities, not relation category, are the error mode that remains.

Future work includes a matched-compute DistMult/GNN comparison, float32 CompGCN-corr, self-adversarial negative sampling, encoding both query and candidate, a ConvE decoder under the same encoder, and moving the Spark stage from `local[*]` to a real cluster if a course-provided YARN or Kubernetes endpoint exists.

Limitations

Training is transductive and English/Wikidata-centric; we do not evaluate multilingual or biomedical downstream tasks beyond `ogbl-biokg` link prediction. Candidate ranking with 500 negatives is not equivalent to full filtered ranking over 90k entities and inflates Hits@k; the OGB evaluator is the fairer protocol and is reported separately. Neighbor sampling with fanout $[10, 10]$ under-represents long-range relational paths. DistMult was trained for an order of magnitude more steps than the GNNs, so Table 3 is not a matched-FLOP study. Query-side GNN scoring never contextualizes the candidate. CompGCN-corr was not re-run in float32 after the AMP failure. Ablations of dimension, depth, bases, and negative count were not executed. Spark runs on one node: Table 5 measures data-size scaling of a local plan, not a multi-executor cluster. Wikidata5M subset training was implemented but not the source of these tables.

Ethics Statement

`ogbl-biokg` contains biomedical entity types; link-prediction scores must not be read as clinical evidence. Wikidata5M inherits Wikidata's coverage and notability biases. We do not release private data. All reported digits come from the executed Kaggle notebook's `results.json`; smoke-test metrics from a one-epoch debug run are not used.

Acknowledgments

This report follows the ACL 2023 proceedings format (A4, two-column, Times, author–year citations, and a mandatory Limitations section). Experiments ran on a Kaggle Tesla T4. Source code is in the public GitHub repository named after this project's title.

References

Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*.

Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D knowledge graph embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Matthias Fey and Jan E. Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428*.

Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*.

Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.

Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. 2016. A review of relational machine learning for knowledge graphs. *Proceedings of the IEEE*, 104(1):11–33.

Andrea Rossi, Denilson Barbosa, Donatella Firmani, Antonio Matinata, and Paolo Merialdo. 2021. Knowledge graph embedding for link prediction: A comparative analysis. *ACM Transactions on Knowledge Discovery from Data*, 15(2):1–49.

Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC)*.

Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*.

Komal Teru, Etienne Denis, and Will Hamilton. 2020. Inductive relation prediction by subgraph reasoning. In *Proceedings of the 37th International Conference on Machine Learning*.

Kristina Toutanova and Danqi Chen. 2015. Observed versus latent features for knowledge base and text inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, pages 57–66.

Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. Complex embeddings for simple link prediction. In *Proceedings of the 33rd International Conference on Machine Learning*.

Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. 2020. Composition-based multi-relational graph convolutional networks. In *International Conference on Learning Representations*.

Xiaozhi Wang, Tianyu Gao, Zhaocheng Zhu, Zhengyan Zhang, Zhiyuan Liu, Juanzi Li, and Jian Tang. 2021. KEPLER: A unified model for knowledge embedding and pre-trained language representation. *Transactions of the Association for Computational Linguistics*, 9:176–194.

Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2015. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations*.

Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65.

A Hyperparameters and paths

Setting	T4 value used
GPU	Tesla T4, 15.6 GB
Dataset	`ogbl-biokg`
`QUICK_RUN` / budget	False / `fast`
Embedding dim	128
GNN layers / R-GCN bases	2 / 8
Batch size	1,024
Fanout	$[10, 10]$
GNN negatives / steps $\times$ epochs	16 / 400 $\times$ 8
DistMult negatives / steps $\times$ epochs	32 / 1,500 $\times$ 20
Label smoothing / dropout	0.1 / 0.2
Adam lr / weight decay / clip	10^{-3} / 10^{-5} / 1.0
Eval triples / candidates	8,000 / 500
Spark driver / partitions	12 g / 8

Table 6: Hyperparameters of the reported T4 run. An RTX 4050 preset exists in the same notebook (dim 64, batch 256, fanout $[8, 8]$, CompGCN-mult only) but was not the source of these tables.

Kaggle: Accelerator = GPU T4, Internet on for the first install and OGB download. Outputs land under `/kaggle/working/outputs` and `/kaggle/working/figures`. Figures in this PDF are those files, copied into `report/figures/`. Length target is 8–10 pages including figures and references (assignment rule); ACL 2023 geometry is preserved.